GraphTransLWG

Tejas Sharma (22B0909) Eshaan Pareek (22B3986)
Matam Kushaal (23B1290)
Aditya Vema Reddy Kesari(22b3985)

Spring 2026

Link to repo

Contents

1 Introduction **1**

2 Problem Formulation **2**

3 Literature Review **2**

Abstract

Transformers enable capturing information across tokens in a sequence, and transforming their embeddings, enabling guesswork of features of the whole sequence or addition of a new token. Adapted to graphs, `GraphTrans` incorporates Graph Neural Netowrks for learning embeddings of nodes that are a function of its neighbors and nodes a few hops away on the graph, along with transformers that treat each node the way a transformer treats a token (but without any positional embeddings, that is, pairwise interaction of all nodes for attention). The graph transformer adds a `[CLS]` token for analyzing the whole graph and classifying it, useful in computer vision and classification of molecules. We propose an architecture that involves multiple small GNNs interleaved with layers of the graph attention layers, with selective attention to neighbors at $\leq k$ hops at earlier attention layers and the GNNs containing MLP-like modules as part of them.

1 Introduction

GNNs suffer from being unable to capture long-range dependencies in graphs. Multiple attempts have been made in the past to solve this problem, such as introducing CNN-like pooling layers, and increasing the number of layers in

the GNNs itself in order to capture longer dependencies. However, these approaches suffer from suboptimal performance due to oversmoothing and unavailability of a universally well-performing pooling operation. The approach used by `GraphTrans`[5] to solve this issue is to use a GNN sub-module followed by a transformer sub-module implementing position-agnostic attention to learn the readout embedding of a graph. The motivation behind doing so is the following: while the GNN learns local dependencies, the attention layers in the transformer enrich the embeddings with global information, using all-pair attention interactions instead of edge-constrained attention. This architecture outperforms the SOTA benchmarks in a variety of applications, including molecule analysis and computer program analysis.

2 Problem Formulation

We upgrade the `GraphTrans` [5] module to an architecture involving multiple GNN layers interleaved with the layers of the transformer [3]. An implementation is already done in [2]. We implement ours differently:

- A GNN module with fewer ($L_{GNN} = 2$) layers is added before each attention layer of the transformer.
- At the earlier layers of the transformer, attention is restricted to within k hops. We consider $k = 3$ for the first layer and $k = 5$ for the second. This is done using an attention mask. At later layers, full-graph attention is done.

We think that the architecture of `GraphTrans` suffers from being unable to capture dependencies on neighbors after multiple attention layers, which is solved by the interleaved GNN layers in our architecture.

Further, in our architecture, earlier layers of attention capture only interactions of a node with its neighborhood as against the entire graph, preventing dilution of information via attention. That is, the attention mask is chosen as $\mathbf{A}^k$ where $\mathbf{A}$ is the adjacency matrix of the graph.

The GNN internally captures the embedding of the node $\vec{h}_{l-1}(u)$, its neighbors $v \in \mathcal{V}(u)$ and the embeddings of the connecting edges $\vec{e}_{l-1}(u, v)$.

We add the `[CLS]` node at the start and attend it to all nodes across all layers of the graph, and use that to classify the whole graph.

We train and test this model in the NC1 and NC109 datasets from [4], the `molpcba` and the `code2` datasets from OGB [1] (open graph benchmarks). We train and test this approach on multiple datasets and compare them to `GraphTrans`.

3 Literature Review

Several works have explored methods related to the problem addressed in this project.

Paper 1: Representing Long-Range Context for Graph Neural Networks with Global Attention [5] Wu et al. propose the GraphTrans architecture that combines a GNN with a Transformer module to better capture long-range dependencies in graphs. The key insight they use is that making the attention computation position-agnostic and allowing all-pairs attention interaction in the transformer enables the model to learn information about far-off pairs of nodes as well that regular GNNs lose out on. This enables GraphTrans to achieve better results than leading models in a variety of applications with negligible time and parameter costs.

Paper 2: Recipe for a General, Powerful, Scalable Graph Transformer[2] Rampášek et al. propose GraphGPS, a hybrid graph neural network architecture that combines local message passing with global self-attention to capture both neighborhood-level and long-range dependencies in graphs. Each GPS layer contains two parallel components: a Message Passing Neural Network (MPNN) that aggregates information from neighboring nodes using real edges, and a Transformer-style global attention module that allows every node to attend to all other nodes in the graph. The outputs of these two modules are combined and passed through an MLP with residual connections and normalization to update node representations.

References

[1] Weihua Hu et al. "Open Graph Benchmark: Datasets for Machine Learning on Graphs". In: *arXiv preprint arXiv:2005.00687* (2020).

[2] Ladislav Rampášek et al. *Recipe for a General, Powerful, Scalable Graph Transformer*. 2023. arXiv: `2205.12454 [cs.LG]`. URL: `https://arxiv.org/abs/2205.12454`.

[3] Ashish Vaswani et al. *Attention Is All You Need*. 2023. arXiv: `1706.03762 [cs.CL]`. URL: `https://arxiv.org/abs/1706.03762`.

[4] Nikil Wale, Ian A Watson, and George Karypis. "Comparison of descriptor spaces for chemical compound retrieval and classification". In: *Knowledge and Information Systems* 14.3 (2008), pp. 347–375.

[5] Zhanghao Wu et al. *Representing Long-Range Context for Graph Neural Networks with Global Attention*. 2022. arXiv: `2201.08821 [cs.LG]`. URL: `https://arxiv.org/abs/2201.08821`.